

LAPORAN PORTOFOLIO TUGAS 6

UNJUK KERJA

Mengimplementasikan Pemrograman Berorientasi Objek
Kode Unit: J.620100.018.02

IDENTITAS MAHASISWA:

Nama Lengkap : Fahra Nusilta Febrianty

NIM : 17240407

Kelas : 17.4A.04

Projek Kasus : Aplikasi Kasir & Booking Servis Bengkel Motor (BengkelPro)

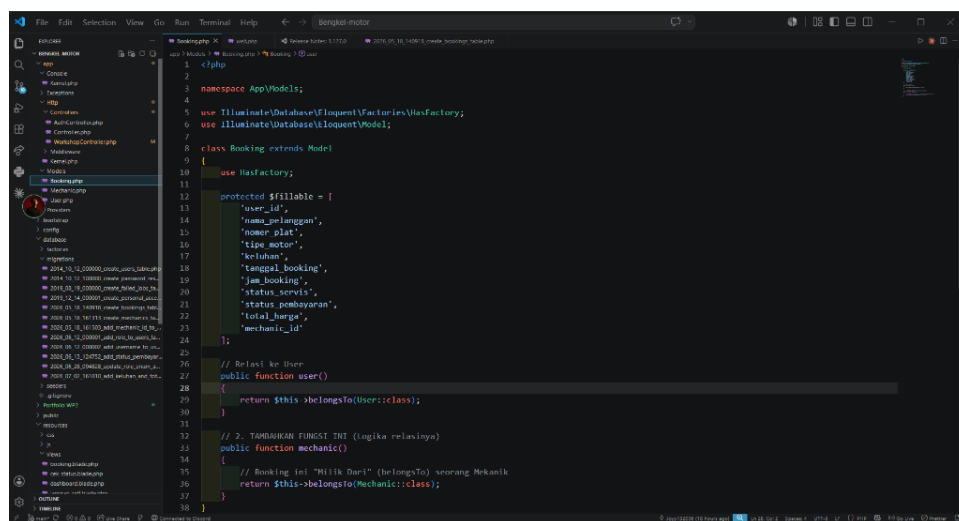
Prinsip OOP 1: Model (Object-Oriented Data Access)

Representasi objek data logis tabel database MySQL menggunakan Eloquent ORM.

```
namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Booking extends Model {
    // Class properties and methods
}
```

Penjelasan Penerapan: Di Laravel, setiap tabel direpresentasikan sebagai sebuah Kelas Model. Interaksi database dilakukan melalui instansiasi objek model, menepis perlunya query SQL manual kasar.



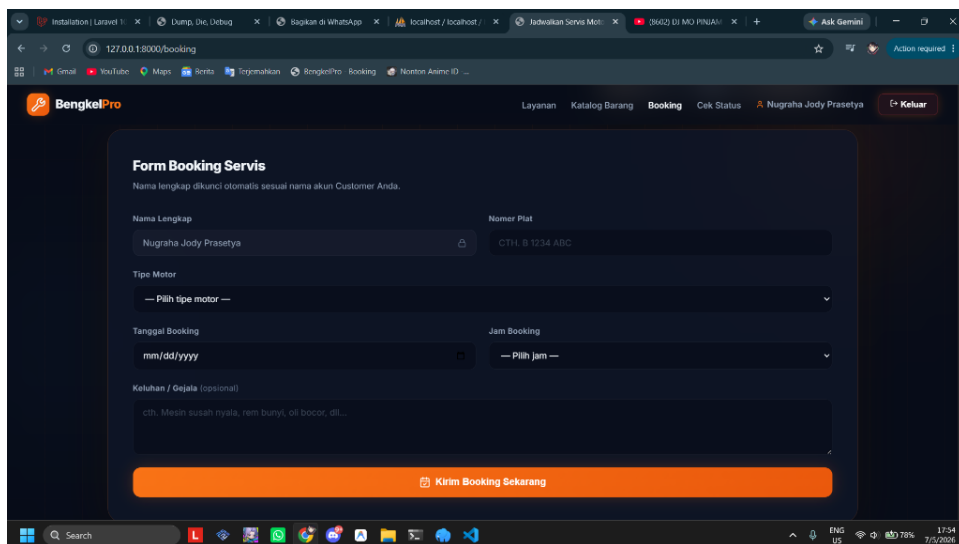
(Screenshot: Model Booking.php di editor VS Code)

Prinsip OOP 2: View (Interface Rendering)

Mengatur visual antarmuka (UI) menggunakan template engine Blade Laravel.

```
<!-- View Template Blade -->
<form action="{ route('booking.simpan') }}" method="POST">
    @csrf
    <input type="text" name="nomer_plat">
</form>
```

Penjelasan Penerapan: View memisahkan kode markup HTML dari logika bisnis controller dengan menggunakan Blade templates yang dinamis.



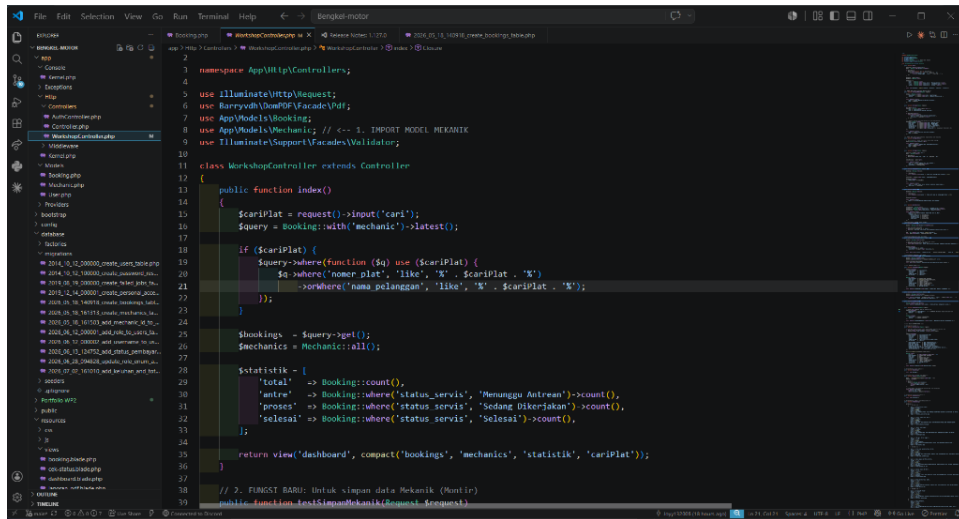
(Screenshot: View booking.blade.php di editor VS Code)

Prinsip OOP 3: Controller (Application Flow Logic)

Kelas pengontrol yang menjembatani data model dan rendering view.

```
class WorkshopController extends Controller {
    public function index() {
        $bookings = Booking::all();
        return view('dashboard', compact('bookings'));
    }
}
```

Penjelasan Penerapan: Controller bertanggung jawab menangani request HTTP, memproses parameter masukan melalui request object, memanggil logika model, dan mengembalikan respon HTML/JSON.



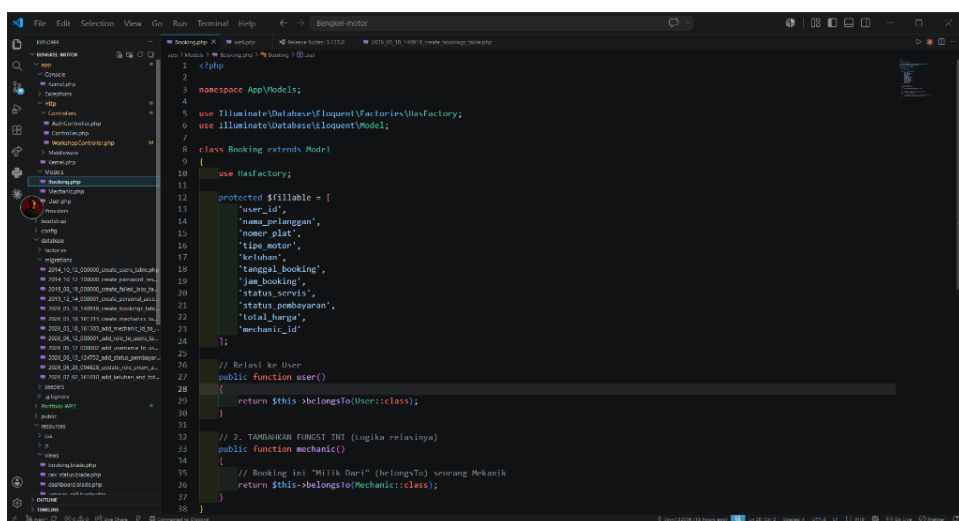
(Screenshot: WorkshopController.php di editor VS Code)

Prinsip OOP 4: Prinsip Encapsulation (Enkapsulasi)

Membungkus data dan membatasi akses properti langsung menggunakan visibilitas protected/private.

```
protected $fillable = [
    'nama_pelanggan', 'nomer_plat', 'tipe_motor', 'status_servis'
];
```

Penjelasan Penerapan: Menggunakan properti \$fillable bermakna data-mass-assignment dibatasi dan diawasi secara ketat oleh enkapsulasi internal model Laravel, mencegah celah security perubahan kolom database ilegal.



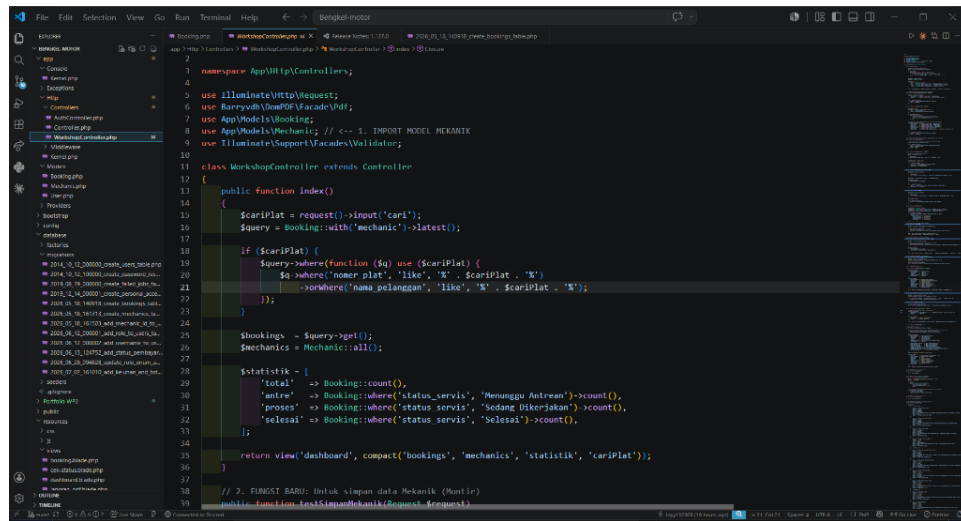
(Screenshot: Enkapsulasi protected \$fillable pada Booking.php)

Prinsip OOP 5: Prinsip Inheritance (Pewarisan)

Mewarisi sifat, method, dan properti dasar dari Class Induk (Core Framework Class).

```
class Booking extends Model { ... }  
class WorkshopController extends Controller { ... }
```

Penjelasan Penerapan: Dengan mewarisi (extends) kelas Model dan Controller bawaan Laravel, sub-class di proyek kita langsung memiliki kapabilitas database query, validator request, dan handler response secara gratis.



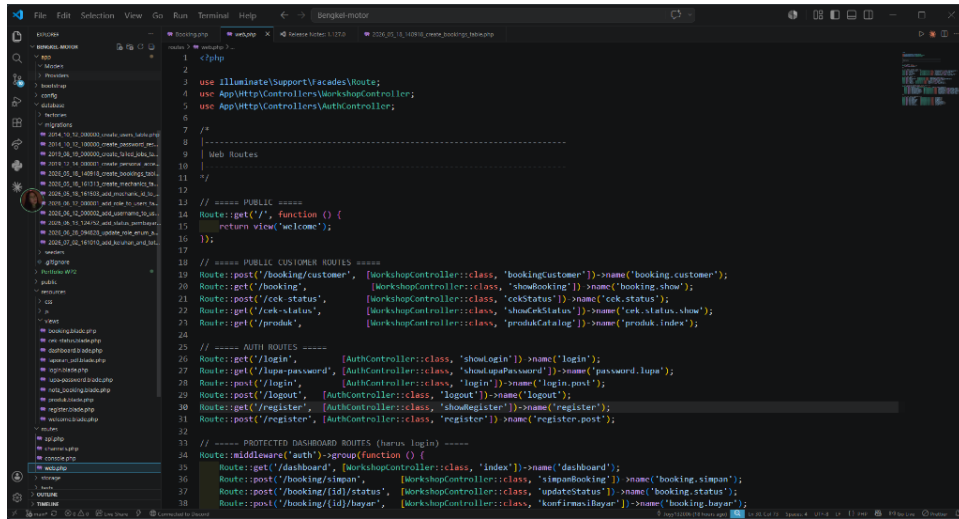
(Screenshot: Inheritance kelas WorkshopController mewarisi Controller)

Prinsip OOP 6: Prinsip Abstraction (Abstraksi)

Menyembunyikan kerumitan detail instruksi sistem di balik antarmuka API yang sederhana.

```
Route::post('/booking/customer', [WorkshopController::class,  
    'bookingCustomer']);  
Schema::create('bookings', function (Blueprint $table) { ... });
```

Penjelasan Penerapan: Abstraksi pada routing menyembunyikan kerumitan dispatching HTTP request dari web server. Abstraksi pada skema migration menyembunyikan sintaks SQL spesifik (seperti CREATE TABLE) di balik Blueprint builder PHP.



```
1 <?php
2
3 use Illuminate\Support\Facades\Route;
4 use App\Http\Controllers\WorkshopController;
5 use App\Http\Controllers\AuthController;
6
7 /*
8 |-----|
9 | Web Routes
10 |-----|
11 */
12
13 // ===== PUBLIC =====
14 Route::get('/', function () {
15     return view('welcome');
16 });
17
18 // ===== PUBLIC CUSTOMER ROUTES =====
19 Route::post('/booking/customer', [WorkshopController::class, 'bookingCustomer'])->name('booking.customer');
20 Route::get('/booking', [WorkshopController::class, 'showBooking'])->name('booking.show');
21 Route::post('/cek-status', [WorkshopController::class, 'cekStatus'])->name('cek.status');
22 Route::get('/cek-status', [WorkshopController::class, 'showCekStatus'])->name('cek.status.show');
23 Route::get('/produk', [WorkshopController::class, 'produkIndex'])->name('produk.index');
24
25 // ===== AUTH ROUTES =====
26 Route::get('/login', [AuthController::class, 'showLogin'])->name('login');
27 Route::get('/lupa-password', [AuthController::class, 'showLupaPassword'])->name('password.lupa');
28 Route::post('/login', [AuthController::class, 'login'])->name('login.post');
29 Route::post('/logout', [AuthController::class, 'logout'])->name('logout');
30 Route::get('/register', [AuthController::class, 'showRegister'])->name('register');
31 Route::post('/register', [AuthController::class, 'register'])->name('register.post');
32
33 // ===== PROTECTED DASHBOARD ROUTES (harus login) =====
34 Route::middleware('auth')->group(function () {
35     Route::get('/dashboard', [WorkshopController::class, 'index'])->name('dashboard');
36     Route::post('/booking/simpan', [WorkshopController::class, 'simpanBooking'])->name('booking.simpan');
37     Route::post('/booking/status', [WorkshopController::class, 'updateStatus'])->name('booking.status');
38     Route::post('/booking/idi/bayar', [WorkshopController::class, 'konfirmasiBayar'])->name('booking.bayar');
```

(Screenshot: Abstraksi routing di routes/web.php)